



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #2

INTERACTION WITH THE “PHYSICAL” SYSTEM

2022 - 2023

Interaction with physical system

- ☐ Revising the kit/simulator “compact storage”
- ☐ Reading sensors & using the actuators
- ☐ Development of kit interaction routines

Run Project: “labwork1”



Don't FORGET...

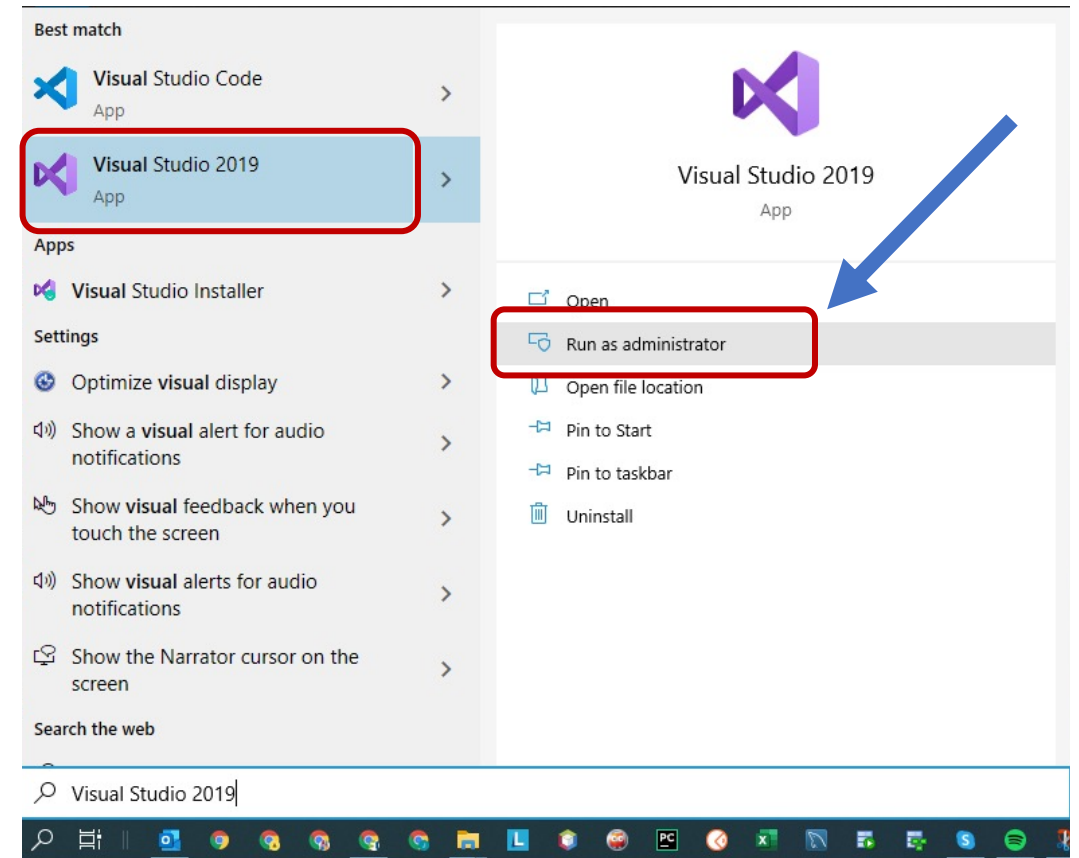


□ Run VS19

- Find Visual Studio 2019 in your startup menu
- Right click with the mouse
- Select “Run as administrator” (*)

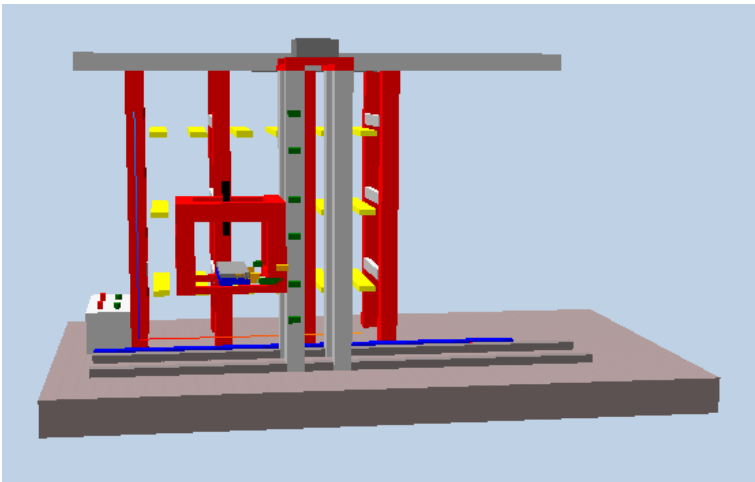
(*) This ensures that our project will obtain more privileges from the Operating System (Windows), ensuring reliable execution in a real time context.

This is important!



- Remember last example from previous class

```
#include<conio.h>
#include<stdlib.h>
#include <windows.h> //for Sleep function
extern "C" {
    #include <interface.h>
}
```



```
int main(int argc, char **argv) {
    createDigitalInput (0);
    createDigitalInput (1);
    createDigitalOutput(2);
    UInt8 p0 = 0;
    UInt8 p2 = 0;
    writeDigitalU8(2, 0x80); // start moving right
    while (!_kbhit()) { //do until a key is pressed
        p0 = readDigitalU8(0);
        p2 = readDigitalU8(2);
        if (!(p0 & (1 << 0))) { // reached right side?
            p2 &= ~(1 << 7);
            p2 |= (1 << 6);
            writeDigitalU8(2, p2); // move left
        }
        if (!(p0 & (1 << 2))) { // reached left side?
            p2 &= ~(1 << 6);
            p2 |= (1 << 7);
            writeDigitalU8(2, p2); //move right
        }
    }
    writeDigitalU8(2, 0);
    closeChannels();
}
```

```
int main(int argc, char **argv) {
    createDigitalInput (0);
    createDigitalInput (1);
    createDigitalOutput(2);
    uInt8 p0 = 0;
    uInt8 p2 = 0;
    writeDigitalU8(2, 0x80); // start moving right
    while (!kbhit()) { //do until a key is pressed
        p0 = readDigitalU8(0);
        p2 = readDigitalU8(2);
        if (!(p0 & (1 << 0))) { // reached right side?
            p2 &= ~(1 << 7);
            p2 |= (1 << 6);
            writeDigitalU8(2, p2); // move left
        }
        if (!(p0 & (1 << 2))) { // reached left side?
            p2 &= ~(1 << 6);
            p2 |= (1 << 7);
            writeDigitalU8(2, p2); //move right
        }
    }
    writeDigitalU8(2, 0);
    closeChannels();
}
```

□ Example for reading sensors values:

Reading x position:

```
uInt8 p = readDigitalU8(0);
if (!getBitValue(p, 2))
    printf("\n is at position 1);
```

How to do this?

Reading from port 0,
the value of bit 2

- Example for reading sensors values:

Reading x position:

```
uInt8 p = readDigitalU8(0);  
if (!getBitValue(p, 2))  
    printf("\n x is at position 1);
```

Copy this function into your project:

```
int getBitValue(uInt8 value, uInt8 bit_n)  
// given a byte value, returns the value of its bit n  
{  
    return(value & (1 << bit_n));  
}
```

- ❑ Example for activating actuators:

Moving to the right:

```
uInt8 p = readDigitalU8(2); //read port 2
setBitValue(&p, 6, 0);      //set bit 6 to low level
setBitValue(&p, 7, 1);      //set bit 7 to high level
writeDigitalU8(2, p);       //update port 2
```

Copy this function into your code:

```
void setBitValue(uInt8 *variable, int n_bit, int new_value_bit)
// given a byte value, set the n bit to value
{
    uInt8 mask_on = (uInt8)(1 << n_bit);
    uInt8 mask_off = ~mask_on;
    if (new_value_bit) *variable |= mask_on;
    else                *variable &= mask_off;
}
```

- ❑ It force us to remember each port, each bit and corresponding functionality, e.g., axis positions, axis movement,... increasing complexity
- ❑ Unsuitable for large systems (e.g., manufacturing systems with thousands sensors /actuators).
- ❑ A good solution is to use a more high-level/"functional" approach, which allows abstraction from the ports, addresses, bits, ... of the many elements in a system

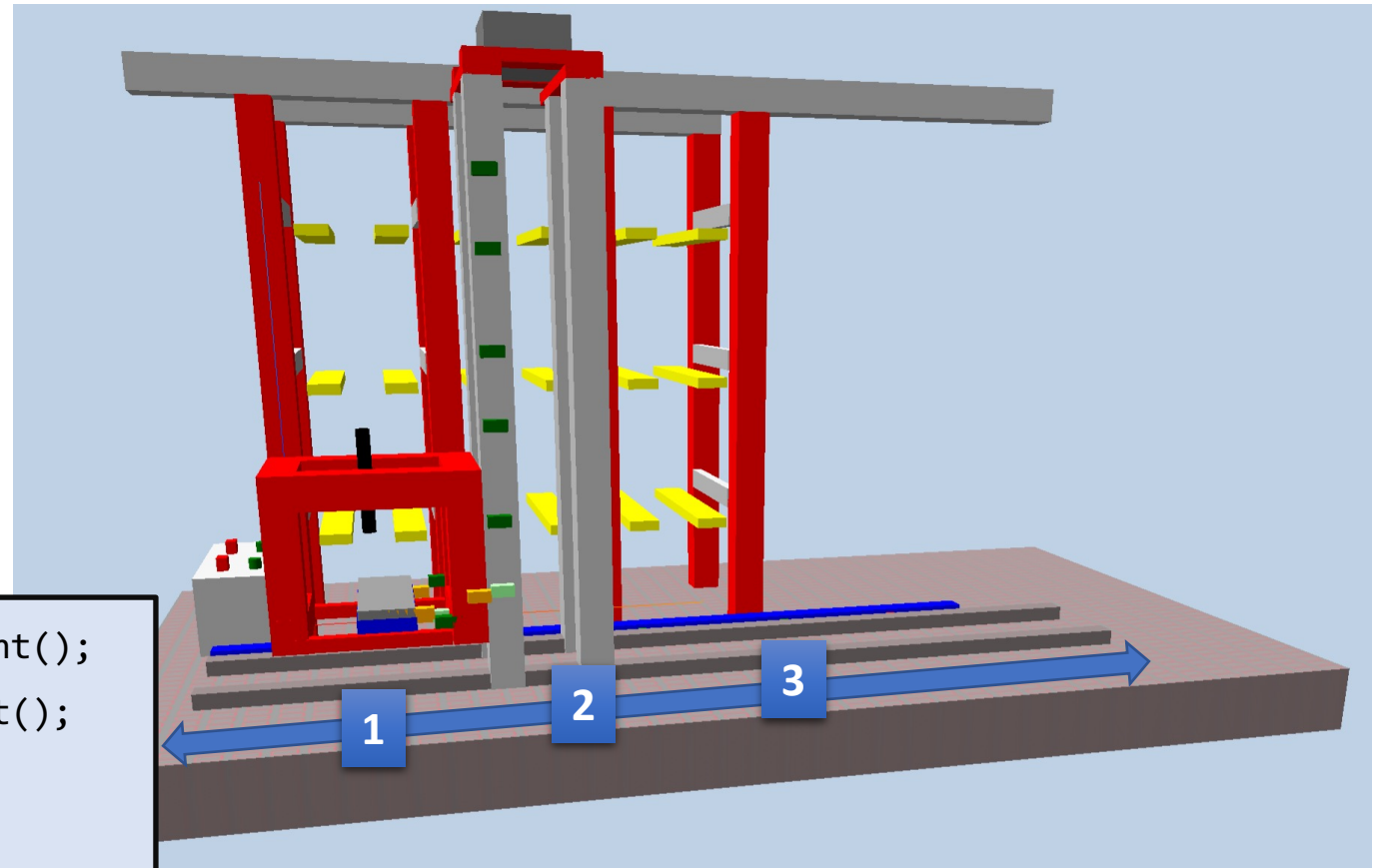
SOME ABSTRACT FUNCTIONALITIES (X AXIS)

Interface.c

```
void createDigitalInput (uInt32 c)  
void createDigitalOutput (uInt32 c)  
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)  
void setBitValue(variable, bit, value)
```

```
void moveXRight();  
void moveXLeft();  
void stopX()  
int getXPos()  
void gotoX(int x)
```



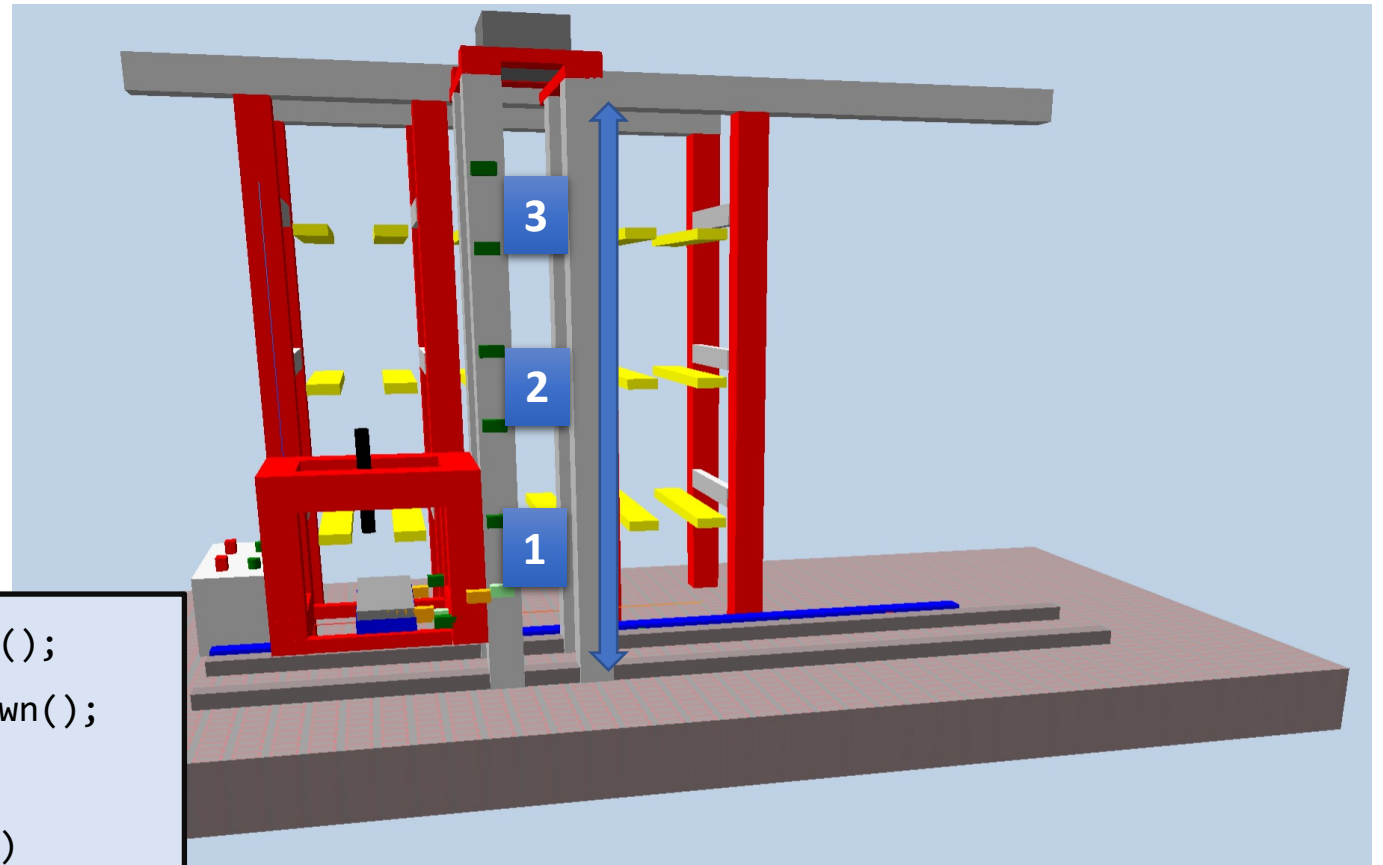
SOME ABSTRACT FUNCTIONALITIES (Z AXIS)

Interface.c

```
void createDigitalInput (uInt32 c)  
void createDigitalOutput (uInt32 c)  
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)  
void setBitValue(variable, bit, value)
```

```
void moveZUp();  
void moveZDown();  
void stopZ()  
int getZPos()  
void gotoZ(int z)
```



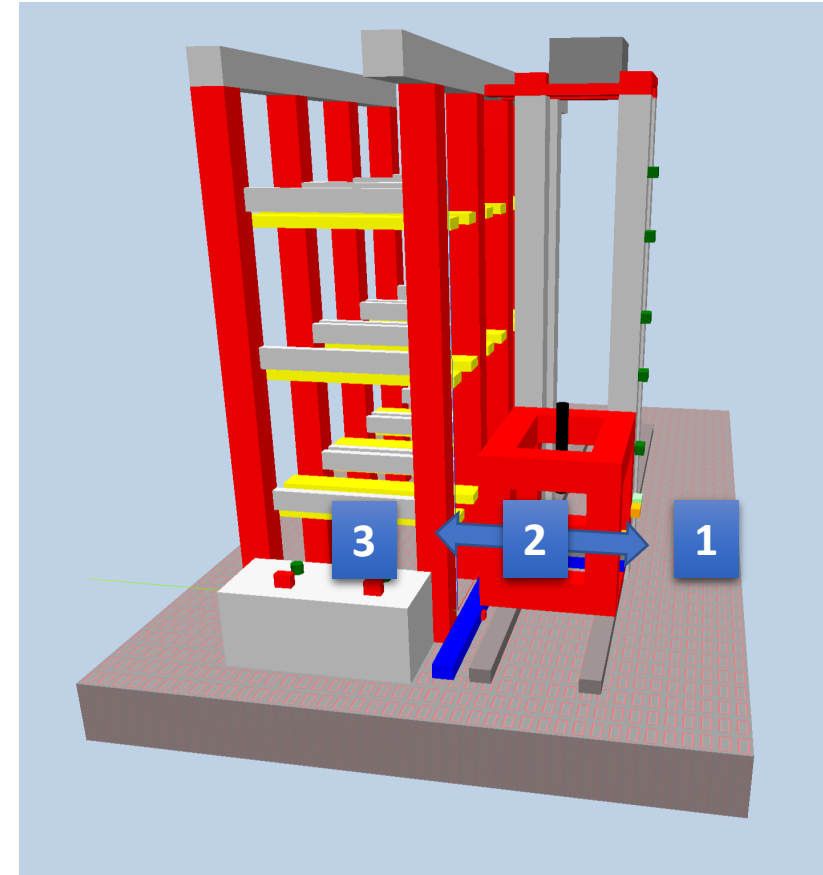
SOME ABSTRACT FUNCTIONALITIES (Y AXIS)

Interface.c

```
void createDigitalInput (uInt32 c)  
void createDigitalOutput (uInt32 c)  
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)  
void setBitValue(variable, bit, value)
```

```
void moveYInside();  
void moveYOutside();  
void stopY();  
int getYPos()  
void gotoY(int y)
```



SOME ABSTRACT FUNCTIONALITIES (OTHERS)

Interface.c

```
void createDigitalInput (uInt32 c)
void createDigitalOutput (uInt32 c)
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)
void setBitValue(variable, bit, value)
```

```
void moveXRight();
void moveXLeft();
void stopX()
int getXPos()
void gotoX(int x)
```

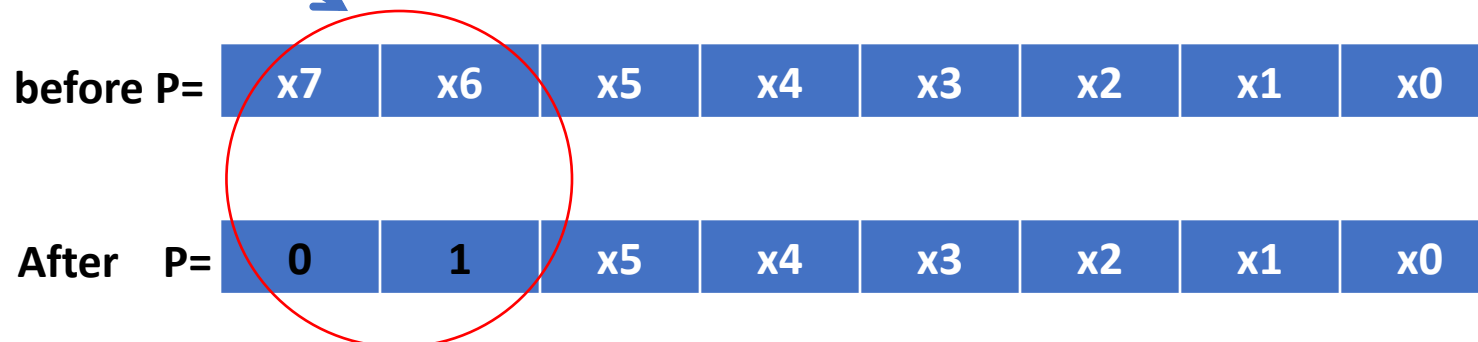
```
void moveZUp();
void moveZDown();
void stopZ()
int getZPos()
void gotoZ(int z)
```

```
void moveYInside();
void moveYOutside();
void stopY();
int getYPos()
void gotoY(int y)
```

```
gotoZUp()
gotoZDown()
putPartInCell()
takePartFromCell()
```

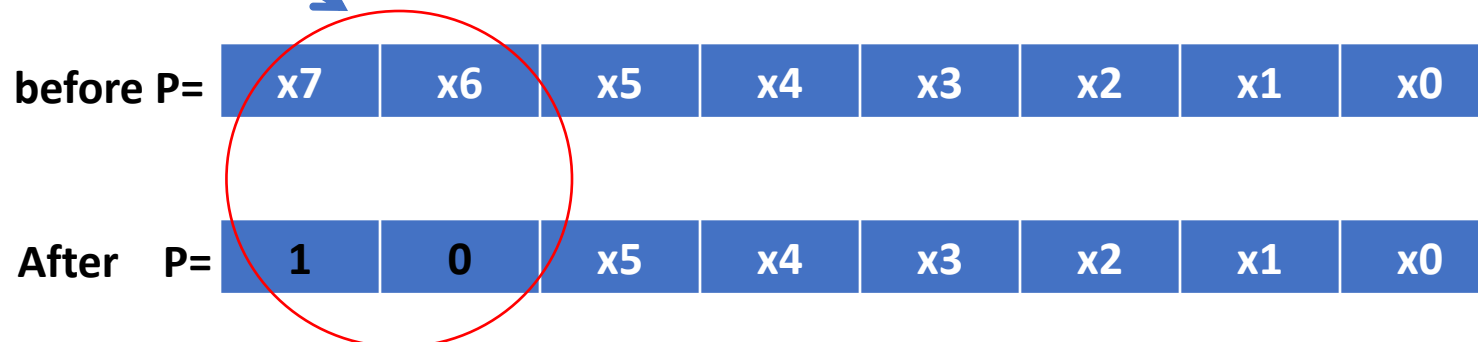
□ Function [moveXLeft\(\)](#) [similar for Z and Y]

```
void moveXLeft()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 6, 1);      // set bit 6 to high level
    setBitValue(&p, 7, 0);      // set bit 7 to low level
    writeDigitalU8(2, p);       // update port 2
}
```



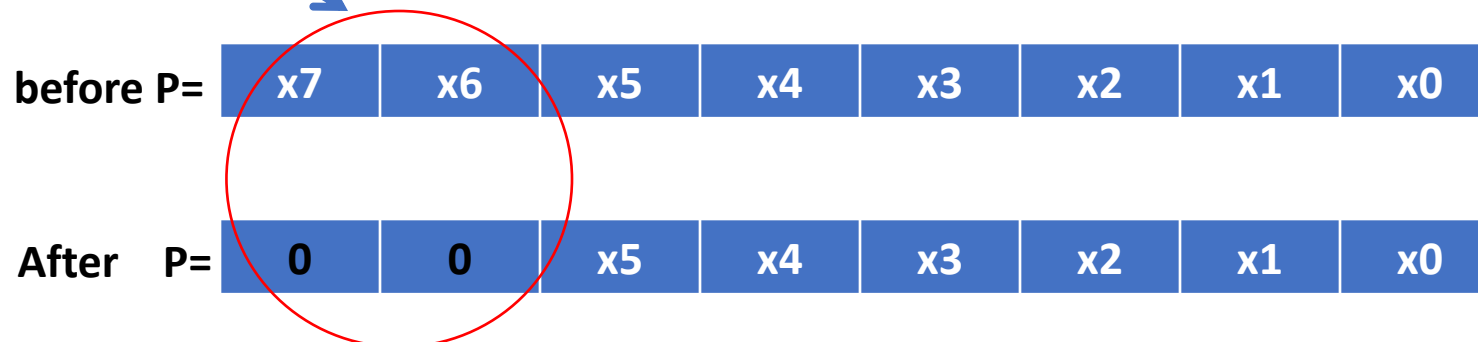
□ Function `moveXRight()` [similar for Z and Y]

```
void moveXRight()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 6, 0);      // set bit 6 to low level
    setBitValue(&p, 7, 1);      // set bit 7 to high level
    writeDigitalU8(2, p);       // update port 2
}
```



- Function `stopX()` [similar for Z and Y]

```
void stopX()
{
    uInt8 p = readDigitalU8(2); // read port 2
    setBitValue(&p, 6, 0);      // set bit 6 to low level
    setBitValue(&p, 7, 0);      // set bit 7 to low level
    writeDigitalU8(2, p);       // update port 2
}
```



☐ Read X, Y and Z positions

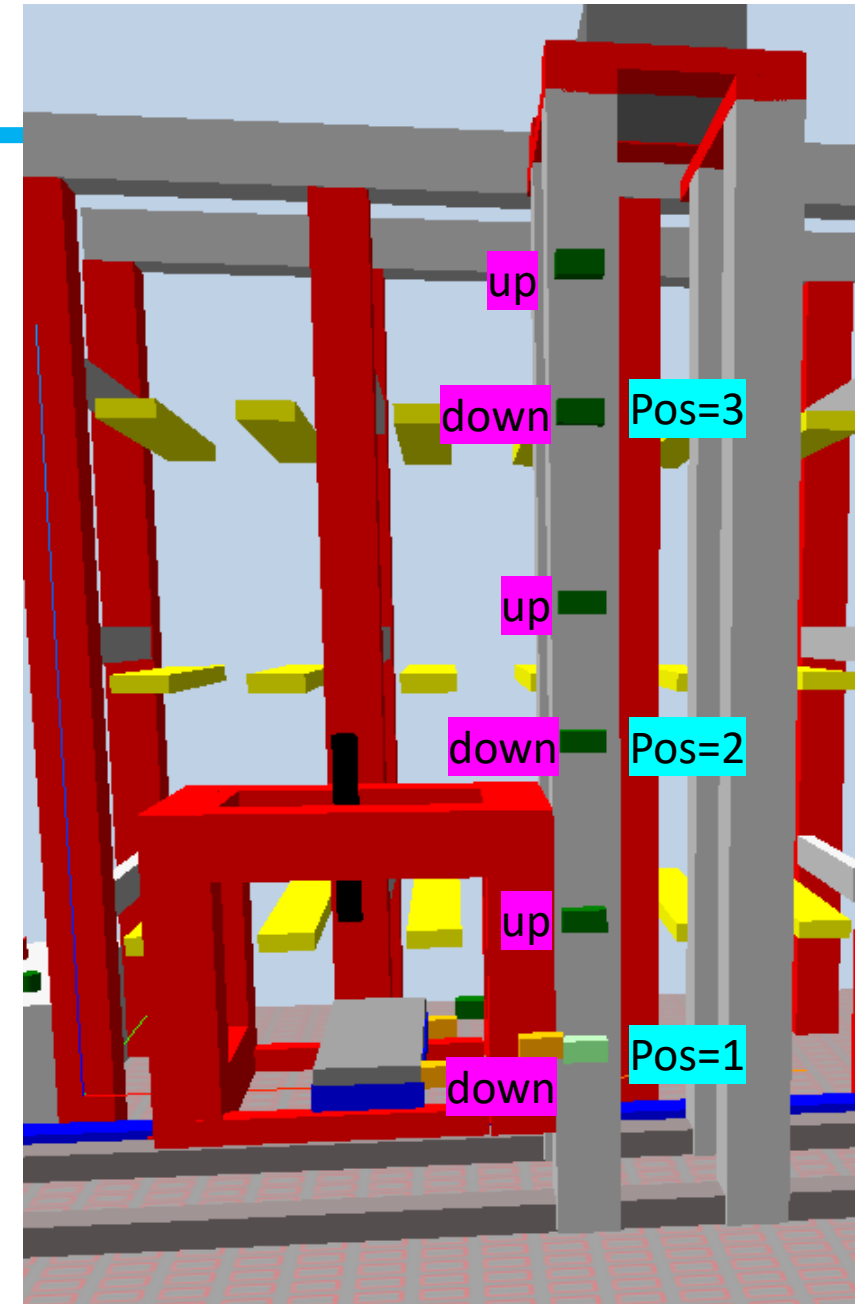
```
int getXPos()
{
    uInt8 p = readDigitalU8(0);
    if( !getBitValue(p,2) )
        return 1;
    if( !getBitValue(p,1) )
        return 2;
    if( !getBitValue(p,0) )
        return 3;
    return(-1);
}
```

```
int getYPos()
{
    //(...) Students' job
}
```

```
int getZPos()
{
    uInt8 p0 = readDigitalU8(0);
    uInt8 p1 = readDigitalU8(1);
    if( !getBitValue(p1,3) )
        return 1;
    if( !getBitValue(p1,1) )
        return 2;
    if( !getBitValue(p0,7) )
        return 3;
    return(-1);
}
```


Z AXIS PARTICULARITIES

- ❑ Z axis has 3 positions.
- ❑ Each position has a pair of sensors: up_position, down_position
- ❑ Use the down_positions **ONLY** for the vertical **position 1,2 and 3**.
- ❑ up_position and down_position sensors are used in *putPartInCell* and *takePartFromCell*



LABWORK1 PROJECT: Work Assignment

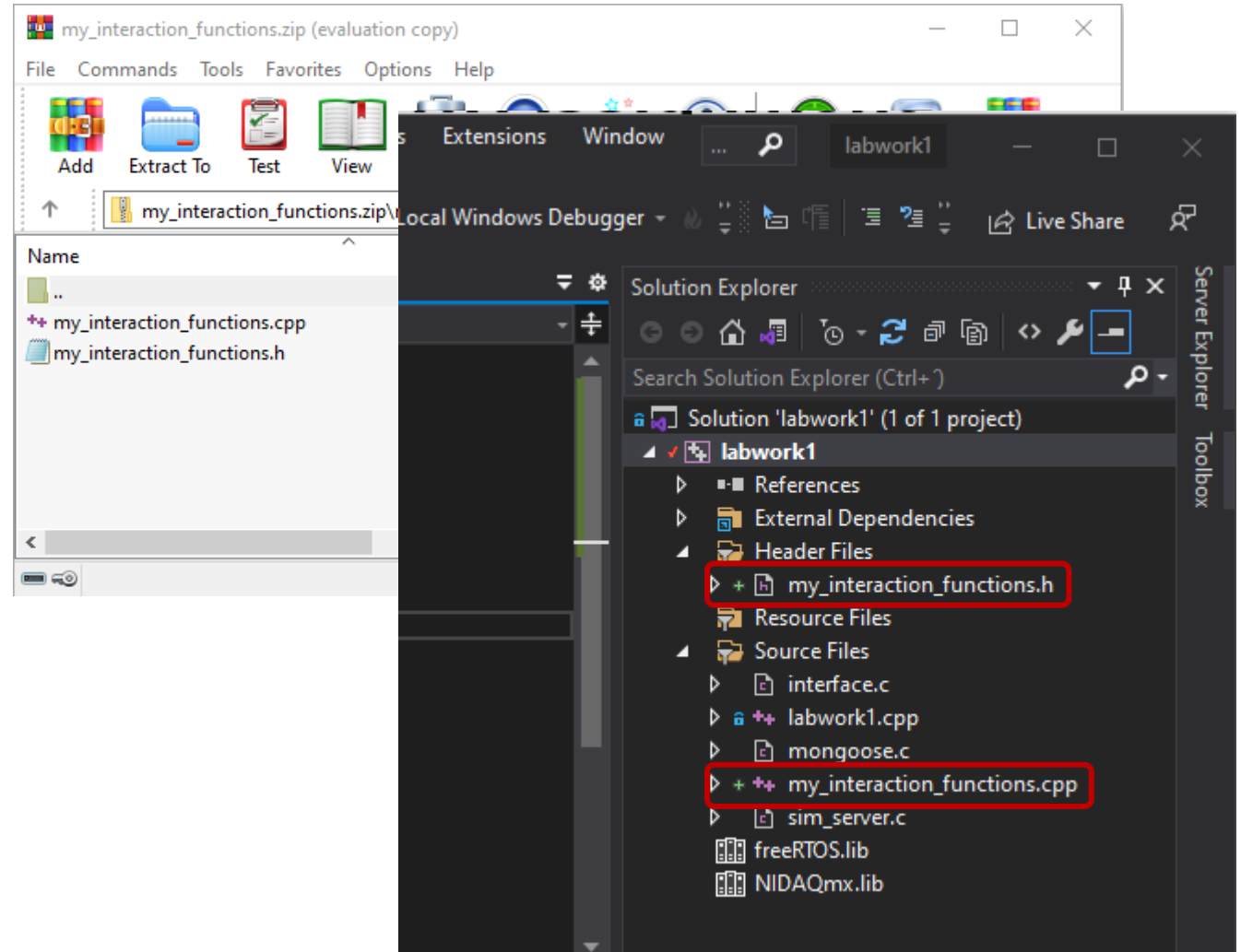


- ❑ Considering the given implementation examples, **develop all the functions specified in the previous slides.**

Suggestion:



- ❑ Extract the **my_interaction_functions.zip** file from CLIP
- ❑ Add the **.cpp** and the **.h** files to the labwork1 project
- ❑ Complete the implementation of the functions



LABWORK1 PROJECT: Work Assignment



Interface.c

```
void createDigitalInput (uInt32 c)
void createDigitalOutput (uInt32 c)
writeDigitalU8(uInt32 port, uInt8 data)
```

```
int getBitValue(value, n_bit)
void setBitValue(variable, bit, value)
```

```
void moveXRight();
void moveXLeft();
void stopX()
int getXPos()
void gotoX(int x)
```

```
void moveZUp();
void moveZDown();
void stopZ()
int getZPos()
void gotoZ(int z)
```

```
void moveYInside();
void moveYOutside();
void stopY();
int getYPos()
void gotoY(int y)
```

```
gotoZUp()
gotoZDown()
putPartInCell()
takePartFromCell()
```

■ In labwork1.cpp

- Create ports
- Launch and manually calibrate the kit simulator
- Test each function:
gotoX(), gotoZ, ... ,
putPartInCell(), takePartFromCell()

```
int main()
{
    // INPUT PORTS
    createDigitalInput(0);
    createDigitalInput(1);
    // OUTPUT PORTS
    createDigitalOutput(2);

    printf("\ncalibrate kit manually and press enter...");

    int tecla = 0;
    while (tecla != 27) {
        tecla = _getch();
        if (tecla == 'd')
            moveXRight();
        if (tecla == 'a')
            printf("vai mover para esquerda");
            //moveXLeft();
        if (tecla == 's')
            stopX();
        if (tecla == 'w')
            printf("vai mover para cima");
        if (tecla == 'x')
            printf("vai mover para baixo");

        // to complete...

    }

    closeChannels();
    return 0;
}
```



KEEP UP THE
GOOD
WORK!